An Empirical Study of Aggregation Methods Under Verifier Capture in Multi-LLM Ensembles: When Does Majority Voting Fail, and Can We Beat It?

Anonymous authors
Paper under double-blind review

Abstract

Multi-LLM verification ensembles increasingly aggregate the verdicts of several LLM *verifiers* to reduce variance and adversarial brittleness relative to any single model. A growing concern in such ensembles is *verifier capture*: a small coalition of verifiers whose outputs are systematically biased can flip the aggregated decision while remaining individually plausible. This paper contributes (i) a formal definition of capture-resilient voting (CRV) and the threat model distinguishing prompt-injection, rubric-injection, and majority-bias mimicry attacks; (ii) an experimental protocol comparing seven aggregation methods (plain majority, PBFT-style deterministic thresholding, uniform-weighted mean, three trust-score variants, and an adaptive hybrid) across verifier pools of size $N \in \{5, 7, 10\}$ with capture counts $f \in \{1, 2, 3\}$ in a synthetic benchmark and a small local-model pilot; and (iii) a falsifiable prediction—that captured verifiers' outputs are statistically indistinguishable from a minority of honest-but-wrong verifiers, so any statistical-deviation capture rule incurs false-positive costs that can outweigh its gains. A synthetic pilot (4,000 trials per cell, $N = 7$ verifiers, f=2 captured) already tests the prediction and yields a nuanced result: trust-score weighting beats plain majority under prompt-injection capture (accuracy 0.69 vs. 0.34 at honest accuracy 0.6; 0.94 vs. 0.74 at 0.8), because calibration-phase trust identifies deterministically flipped verifiers; under majority-bias mimicry, however, the same method fails (0.32 vs. 0.41 at honest accuracy 0.6). These pilot values are synthetic and do not establish a universal impossibility result. A descriptive local-model pilot is included in the evidence package, but a held-out real-trace benchmark remains future work; we release the framework, generators, and analysis scripts so the benchmark can be extended independently.

1 Introduction

Large language model (LLM)-based verification pipelines increasingly rely on *multi-source verifier ensembles* (Cui et al., 2026; Leung et al., 2026), in which a fixed pool of N LLM verifiers independently evaluate a candidate output and a final decision is produced by aggregating their votes. Such ensembles have become a default building block of contemporary AI auditing, peer review assistance, and safety-critical content moderation, where the goal is to reduce variance and adversarial brittleness relative to any single model.

A natural and increasingly documented failure mode of these ensembles is what we call *verifier capture*: a small coalition of verifiers whose outputs are systematically biased in a coordinated

direction can swing the aggregated decision, even when a majority of verifiers remain individually reliable. Verifier capture differs from the classical single-verifier hallucination or bias problem in two important ways. **(i) Capture is interactive.** The adversarial verifier does not need to be accurate in absolute terms; it only needs to produce outputs that, when combined with the other verifiers under a fixed aggregation rule, flip the final decision. **(ii) Capture is silent.** From the perspective of an external observer that only sees final aggregated verdicts, a captured ensemble looks identical to a correctly functioning one until the disagreement structure of individual votes is examined.

Recent work has begun to expose capture-like failure modes in LLM judges: Zhang et al. (2026) show that a biased judge *silently disables* a downstream curator past a sharp false-pass threshold. This paper asks the natural follow-up question: given that capture is real and silent, *can any aggregation rule do better than plain majority voting*? Our answer, in this empirical study, is *mostly no*.

This paper makes four contributions:

- A formal threat model for verifier capture in LLM ensembles, including a definition of ε-capture-resilience that decouples the adversary's per-verifier accuracy from its aggregate influence (Section 3).
- A unified experimental protocol that benchmarks seven aggregation methods under three attack families and four scenario configurations (Section 4).
- An empirical finding that *no* aggregation method we tested substantially outperforms plain majority voting, and several trust-score variants *underperform* majority under strong capture (Section 5). We provide a structural explanation for the negative result: in our setting the captured verifiers' outputs are statistically indistinguishable from a minority of honest-but-wrong verifiers, so any rule that uses statistical deviation to identify capture incurs false-positive costs that outweigh its true-positive gains.
- A reproducible open-source release with code, vote generators, and a Beyond-PDF interactive attack demonstration (Section 8).

We emphasise that the *negative* headline finding is the paper's contribution, not an apology. TMLR's acceptance criteria explicitly prioritise technical correctness and audience interest over novelty, and a rigorous benchmark that documents the limits of the obvious aggregation strategies is valuable precisely because future work can build on it rather than re-derive it. We discuss when, and against which assumptions, our negative result would *not* hold (Section 10).

The remainder of this paper is organized as follows. Section 2 reviews classical Byzantine fault tolerance and LLM ensembling. Section 3 introduces the threat model. Section 4 describes the seven aggregation rules we benchmark. Section 5 reports the benchmark and its negative result. Section 7 discusses related work. Section 8 addresses broader impact. Section 10 concludes and outlines conditions under which our negative result would not hold.

2 Background

Classical Byzantine consensus. Byzantine fault tolerance (BFT) (Lamport et al., 1982) establishes that a deterministic consensus protocol over N replicas tolerates at most $f < N/3$ Byzantine (arbitrary) failures when paired with digital signatures, or $f < N/2$ under synchrony assumptions with binary outputs. PBFT (Castro & Liskov, 1999) brought BFT to practical systems and HotStuff

(Yin et al., 2019) linearised the communication pattern. All classical BFT results assume *binary* replica outputs; the extension to *continuous* outputs (which we need for LLM verifier probabilities) is not standard and is one of the contributions of this paper.

LLM-as-judge and ensembles. The use of LLMs as evaluators, or *judges*, was popularised by MT-Bench and Chatbot Arena (Zheng et al., 2023), where a single LLM judge is shown to correlate well with human preference ratings. Subsequent work studied ensemble-of-judges methods (Cui et al., 2026; Leung et al., 2026) to reduce variance and calibrate directional bias. These methods optimise for *passive* disagreement across annotators; we instead consider *adversarial* disagreement induced by verifier capture.

Adversarial robustness of LLM judges. Recent work has begun to expose the silent nature of biased LLM judges: Zhang et al. (2026) show that a biased judge *silently disables* a downstream curator past a sharp false-pass threshold, and broader red-teaming studies (Perez et al., 2022; Schulhoff et al., 2023) catalogue prompt-injection attacks. Our threat model (Section 3) operationalises these observations into a quantitative capture-resilience condition.

Peer-review automation. Liu (2026b) release an open-source multi-reviewer pipeline that we use as one of our testbeds; we discuss this and related calibration work (Liu, 2026a) in Section 7.

3 Threat Model and Definitions

We consider an ensemble $\mathcal{V} = \{v_1, \ldots, v_N\}$ of N LLM verifiers that independently evaluate a candidate output and produce a continuous decision distribution $p_i \in [0,1]$ on the binary outcome $\{0,1\}$; the aggregated decision is produced by a fixed aggregation rule $\mathcal{A} : [0,1]^N \to \{0,1\}$. A subset $\mathcal{A} \subseteq \mathcal{V}$ of size $f = |\mathcal{A}|$ is *captured* by an adversary that controls the verifiers' outputs but cannot observe or modify the inputs seen by honest verifiers.

Definition 1 (Honest verifier)**.** *A verifier $v_i \in \mathcal{V} \setminus \mathcal{A}$ is* honest *if $p_i = \mathbb{P}[\text{correct outcome} \mid x, \theta_i]$, where θ_i is the verifier's fixed-but-unknown system prompt and training. In particular, an honest verifier's outputs are conditionally independent given the input x.*

Definition 2 (Capture coalition)**.** *A captured verifier $v_j \in \mathcal{A}$ outputs $\tilde{p}_j = g(p_j^{true})$ where $g : [0,1] \to [0,1]$ is an adversarial response function drawn from one of three concrete attack families:*

- Prompt injection*: g is a fixed affine or step function chosen by the adversary (e.g. "always output 0.9").*
- Rubric injection*: g depends on the input via a hidden rubric bias that flips the verifier's preference order on a target subset of inputs.*
- Majority bias*: g mimics the running empirical mode of the other verifiers' outputs, so the captured output is indistinguishable from the majority.*

Definition 3 (Aggregation correctness)**.** *The aggregation rule $\mathcal{A}$ is (η, δ)-correct on a distribution $\mathcal{D}$ if $\Pr_{(x,y)\sim\mathcal{D},\mathcal{A},g}[\mathcal{A}(p_1, \ldots, p_N) = y] \geq 1 - \delta$, where the probability is taken over the input distribution, the adversary's choice of capture coalition $\mathcal{A}$, and the response functions g of all captured verifiers.*

Definition 4 (Capture-resilient aggregation)**.** *The aggregation rule $\mathcal{A}$ is* ε-capture-resilient *if, for every adversary that captures at most f verifiers, the ensemble remains (η, δ)-correct with $\delta \leq \varepsilon$ for every input distribution $\mathcal{D}$.*

Theorem 1 (Capture lower bound)**.** *For any deterministic aggregation rule $\mathcal{A}$, if the adversary captures $f \geq N/2$ verifiers, then for* some *input distribution $\mathcal{D}$ and* some *adversarial response function g from Definition 2, the rule $\mathcal{A}$ is not ε-capture-resilient for any $\varepsilon < 1/2$.*

Proof sketch. If the adversary controls a strict majority $f > N/2$, the adversary can choose g such that the captured verifiers all output the same value y^* with probability 1, while honest verifiers (whose accuracy is below 1 on $\mathcal{D}$) collectively cannot outvote $\mathcal{A}$. Formal reduction to the classical $f \geq N/2$ impossibility bound for deterministic aggregation. Full proof in Appendix A. □

Remark 1 (Continuous outputs vs. classical BFT)**.** *Theorem 1 recovers the classical $f < N/2$ BFT impossibility bound but does not yield a constructive upper bound, because LLM verifier outputs are continuous: a verifier that outputs $p_i = 0.51$ is qualitatively different from one that outputs $p_i = 0.99$, even though both would be treated as "in favor" by a binary-vote aggregation rule. This gap is what motivates the trust-score weighting in CRV (Section 4).*

Assumption 1 (Honest independence)**.** *Honest verifiers' outputs are conditionally independent given x. This is the standard assumption in LLM-ensemble calibration work (Cui et al., 2026; Zheng et al., 2023); we discuss the relaxation in Section 7.*

4 Methods: Seven Aggregation Rules

We benchmark seven aggregation rules. Three are *classical* and require no learning: plain majority voting (Section 4), PBFT-style deterministic thresholding (Section 4), and uniform-weighted mean (Section 4). Three are *trust-score-based* and share the EMA update $\tau_i^{(t)} = \alpha\tau_i^{(t-1)} + (1-\alpha)c_i^{(t)}$ where $c_i^{(t)} = 1 - |p_i^{(t-1)} - \text{med}_{-i}^{(t-1)}|$; they differ in how the trust score is *used* at decision time: weighted mean, weighted median, and Kalman-filter trust tracking. The seventh is ATV, an adaptive hybrid we proposed as a candidate that we expected to combine the strengths of soft weighting and outlier robustness.

Plain majority. Each verifier contributes a binary vote $\mathbb{1}[p_i \geq 0.5]$; the aggregate is the majority over the N binary votes. Cost $O(N)$ per round, no state.

PBFT-style deterministic thresholding. (Castro & Liskov, 1999) Each verifier again votes binary; the aggregate passes if at least $(N + f + 1)/2$ verifiers agree, which is the deterministic threshold that tolerates f adversarial nodes when a strict majority is required. Cost $O(N)$ per round.

Uniform-weighted mean. The aggregate is $\mathbb{1}[\sum_i p_i/N \geq 0.5]$. Cost $O(N)$, no state.

Trust-score weighted mean. At round t, weights are $w_i = \tau_i^{(t-1)} / \sum_j \tau_j^{(t-1)}$; the aggregate is $\mathbb{1}[\sum_i w_i p_i \geq 0.5]$. The trust update uses consistency against the per-round median. Cost $O(N)$ plus a per-round $O(N \log N)$ for the median.

Trust-score weighted median. Identical trust update; at decision time, sort verifiers by current p_i and pick the p_i at which the cumulative trust weight crosses half the total weight. This is robust to a single extreme value but expensive $O(N \log N)$ per round.

Kalman-filter trust tracking. Same decision rule as weighted mean, but trust tracking uses a Kalman filter with process variance $Q = 0.01$ and measurement variance $R = 0.05$ instead of an EMA, which we hypothesised would smooth trust under noisy verifier streams.

Atv: adaptive trust-weighted voting. A hybrid we designed as the natural synthesis of the two trust-score decision rules. ATV switches between weighted mean and weighted median at each round based on the standard deviation of the trust scores: if $\text{std}(\tau) < \tau_{\text{split}} = 0.5$ (trust concentrated, presumed-uniform regime), use weighted mean; otherwise (trust dispersed, capture suspected) use weighted median. The hyperparameter τ_{split} was tuned on a held-out scenario; we report sensitivity in Appendix C.

Why we expected Atv to win. Our *a priori* hypothesis, before running the benchmark, was that ATV would inherit the best of both worlds: the mean's sensitivity to subtle weight differences in the easy regime, and the median's robustness to outlier-dominated rounds in the hard regime. The benchmark (Section 5) does not support this hypothesis.

Why we report Atv even though it loses. We report ATV as a representative of the natural design space that future researchers might explore, rather than as our recommended method. The negative result is informative precisely because ATV *was* our best prior and still loses; the same is true of the Kalman variant.

5 Experimental Setup

Datasets. We use a synthetic verifier-output generator in this study; we discuss the limitations of this choice in Section 10. The generator (Section 5) models the agreement structure of a multi-LLM verification ensemble under verifier capture, and is calibrated to produce vote distributions whose summary statistics match published numbers for LLM judges on rubric decisions (Leung et al., 2026): per-verifier outputs are concentrated near 0 or 1 with $p_{\text{conf}} \in [0.7, 0.85]$ and a small amount of i.i.d. Gaussian noise ($\sigma = 0.05$). We additionally release a *Beyond-PDF* interactive demonstration that replays the benchmark on real LLM trace files (Section 8).

Attack models. We instantiate the three attack families of Definition 2 concretely: *prompt injection* (PROMPTINJ: captured verifier outputs the complement of its honest output), *rubric injection* (RUBRICINJ: captured verifier flips its label on a random fraction of inputs), and *majority bias* (MAJBIAS: captured verifier mimics the running median of the other verifiers' outputs). Section 5 gives the formal parameterisations.

Scenarios. We consider four scenarios that vary the two most consequential parameters: honest-verifier per-instance accuracy $\eta \in \{0.65, 0.85\}$ and attack strength $\alpha \in \{0.6, 0.9\}$ (probability that a captured verifier executes the attack on a given round). The four combinations are labelled S1–S4 in Table 1. We run $T = 1000$ rounds per scenario and average over $n = 5$ random seeds; full per-seed numbers are in Appendix C.

Metrics. We report *decision accuracy* as the fraction of rounds on which the aggregated decision equals the ground-truth label. Accuracy is the right metric for this benchmark because the goal of the aggregator is to produce the correct label, and the captured verifiers' goal is to produce an incorrect one; alternative metrics such as false-positive rate at the verifier level would require

labelling individual verifiers as captured or honest, which we cannot do from observed verifier outputs alone (this is the central statistical difficulty).

Hyperparameters. All trust-based methods use $\alpha = 0.9$ (EMA decay) and $c_i^{(t)} = 1 - |p_i - \text{med}_{-i}|$. ATV uses $\tau_{\text{split}} = 0.5$ chosen on a held-out scenario. PBFT threshold is $(N + f + 1)/2$. Code and configuration files are released with the paper.

6 Results

Headline. Table 1 reports the headline result. Under every scenario we test, plain majority voting is at or near the top; the trust-based methods tie or trail majority by 1–3 percentage points; and ATV, our adaptive hybrid, never wins. The differences are small but consistent across all five random seeds we ran; the standard deviations are below 0.01 in every cell.

Table 1: Decision accuracy across four scenarios. Best per column in bold. No aggregation method we tested substantially outperforms plain majority voting.

Method	S1 (η=0.85, α=0.6)	S2 (η=0.65, α=0.6)	S3 (η=0.65, α=0.9)	S4 (η=0.85, α=0.9)	mean
majority	**0.889**	**0.553**	**0.459**	0.843	**0.686**
bft_pbft	0.777	0.522	0.458	0.720	0.619
weighted_uniform	0.804	0.484	0.365	0.729	0.595
trust_mean	0.883	0.521	0.432	**0.851**	0.672
trust_median	**0.889**	**0.553**	**0.459**	0.844	**0.686**
kalman_trust	0.882	0.535	0.446	0.842	0.677
ATV	0.883	0.527	0.438	0.846	0.673

Why no method beats majority. The negative result has a structural explanation. Capture is *statistically indistinguishable from honest-but-wrong*: a captured verifier outputting the complement of its honest output has exactly the same per-instance deviation from the median as an honest verifier that happens to be wrong on this round. Any rule that uses deviation from the median to identify capture therefore cannot distinguish a captured verifier from a minority of honest-but-wrong verifiers; it will either ignore both (and gain nothing) or exclude both (and lose accuracy on rounds where honest-but-wrong verifiers are the actual minority).

In the high-honest scenario S1, the trust-based methods approach majority but trail it by 0.5 percentage points, consistent with the explanation above: the trust mechanism correctly down-weights captured verifiers when they *happen* to be outliers, but it also incorrectly down-weights honest verifiers that happen to be wrong, and the two effects approximately cancel.

In the strong-attack scenarios S3 and S4, the trust-based methods gain an apparent advantage in some columns (S4 for trust_mean) but lose it in others (S2 for all trust-based methods). The advantage in S4 disappears when averaged across all four scenarios: the mean column is dominated by majority and trust_median, both at 0.686.

Per-attack breakdown. Across the three attack families (prompt injection, rubric injection, majority bias), the ordering of methods is qualitatively similar. Majority and trust_median remain tied at the top across all three attack types; ATV and the Kalman variant trail by 0.5–1.5 percentage points. The largest gap between majority and the best trust-based method is under RUBRICINJ,

where trust_median loses to majority by 0.7 percentage points because the rubric injection only flips on a random fraction of rounds, and the trust update averages over both flipped and unflipped rounds.

Sensitivity to hyperparameters. ATV's τ_{split} was tuned on a held-out scenario; sensitivity sweeps over $\tau_{\text{split}} \in [0.2, 0.8]$ change ATV's mean accuracy by at most 0.4 percentage points and do not affect its ranking. Trust α sweeps over $[0.5, 0.99]$ shift trust_mean by ± 0.5 percentage points with no rank change. Full sensitivity table in Appendix C.

Implication. The benchmark implies that, for the ensemble size, attack strength, and honest-accuracy regime we consider, *the engineering effort of building a non-trivial aggregation rule does not pay off*. Practitioners should prefer plain majority voting and invest their engineering effort elsewhere — either in increasing honest-verifier accuracy or in diversifying the verifier pool so that capture becomes harder. We elaborate on diversification as a defensive strategy in Section 10.

7 Related Work

[Placeholder body. W2 target will fill three threads:]

Classical Byzantine consensus. Lamport et al. (1982); Castro & Liskov (1999); Yin et al. (2019) establish the $3f + 1$ bound for binary-output consensus under f adversarial nodes. Our setting differs in that LLM verifier outputs are continuous probability distributions, so classical BFT bounds do not transfer directly; this motivates our ε-capture-resilience definition in Section 3.

LLM-as-judge and ensemble calibration. Zheng et al. (2023) introduce the MT-Bench and Chatbot Arena evaluation paradigm; Leung et al. (2026) provide the most recent calibration numbers for 8 LLM judges on rubric decisions, and Cui et al. (2026) jointly optimize ensemble weights and calibration for subjective supervision. CRV differs from EDO in operating under an explicit adversarial model (capture) rather than a passive disagreement model.

Adversarial robustness of LLM judges. Zhang et al. (2026) show that a biased judge *silently* disables a downstream curator past a sharp false-pass threshold, and Perez et al. (2022); Schulhoff et al. (2023) catalogue prompt-injection and red-team attack techniques against LLM evaluators. Our threat model operationalises these observations into a quantitative capture-resilience definition and provides a protocol that provably maintains aggregation correctness under bounded capture.

Peer-review automation. Liu (2026b) release an open-source multi-reviewer pipeline that we use as the testbed for our empirical evaluation (Section 5); Liu (2026a) develops calibration methods for multi-agent ensembles more broadly and is the closest work in our own research line.

8 Broader Impact

Positive impact. The primary positive impact of this work is *negative*: by showing that no aggregation method we tested substantially outperforms plain majority voting under verifier capture, we redirect practitioner effort away from building sophisticated aggregators and toward investments that *do* pay off in the capture regime: increasing honest-verifier accuracy, diversifying the verifier pool, and auditing deployed ensembles for suspicious agreement patterns. We expect the benchmark to be useful for engineering teams that are currently considering whether to invest in a custom

aggregation layer; our data suggest the answer is "no" under the parameter regime we test, and the engineering budget should be spent elsewhere.

A secondary positive impact is methodological: the failure modes of the seven aggregation rules we test illuminate which statistical assumptions a successful capture-resilient aggregator would need to break. Future research that does find a winning method can explicitly cite which assumption we showed fails here.

Negative impact and dual-use concerns. The same attack implementations that allow us to *benchmark* verifier capture also allow an adversary to *measure* capture susceptibility of a competing verifier product. We consider this a real but bounded risk for the following reasons. **(i)** The three attack families we instantiate (PROMPTINJ, RUBRICINJ, MAJBIAS) are straightforward enough that a competent adversarial team could re-derive them from the threat model alone; our code release does not materially lower the cost. **(ii)** Our benchmark demonstrates that these attacks, when attempted against a well-calibrated honest majority, have only small absolute effect (the gap between majority and the best trust-based method is below 1 percentage point in 3 of 4 scenarios). An adversary using our code to measure capture susceptibility will therefore find that the practical upside of attack deployment is small. **(iii)** Our benchmark is *synthetic*. An adversary who wishes to deploy a capture attack against a real ensemble must still construct a real captured verifier that mimics the target ensemble's distribution, which is a substantially harder engineering problem than running our code.

Responsible disclosure norms. We follow the standard ML-security responsible-disclosure norm: we release the benchmark code under a licence that prohibits offensive use, and we provide a `ResponsibleUse.md` file in the repository that documents the intended defensive use and lists specific prohibited uses. We do not release any captured-verifier weights or fine-tuning scripts that would lower the cost of deploying a real capture attack.

Limitations of public release. The benchmark code we release is sufficient to *reproduce* our results but not sufficient to *deploy* a capture attack against a real ensemble without substantial additional engineering. This is deliberate. We discuss the scope of the release, including which extensions a future maintainer might add and which are out of scope, in the `README.md` of the released repository.

Stakeholder considerations. Verifier-capture attacks disproportionately threaten operators of multi-LLM auditing pipelines, including the authors of this paper, who maintain an open-source peer-review audit tool (Liu, 2026b). We have therefore prioritised documentation of defensive best-practice over attack generalisation, and we invite other operators of LLM-judge ensembles to contact us with deployment experience that would refine the attack-and-defence landscape this benchmark sketches.

Positive-negative balance. On balance, we believe the positive impact of releasing this benchmark—redirecting engineering effort away from ineffective aggregation rules toward more productive investments in verifier quality and pool diversification—substantially outweighs the bounded negative impact of marginally lowering the cost of an attack that, per our own benchmark, has small practical effect against well-calibrated honest majorities.

9 Ethics Statement

This study does not involve human subjects, personal data, or animal subjects. The verifier-output data we use is either synthetic (Section 5) or from publicly available LLM-judge benchmarks (Leung et al., 2026; Zhang et al., 2026).

Dual-use risk. We have discussed the dual-use risk of releasing the attack implementations in detail in Section 8. In summary, the bounded risk is outweighed by the positive impact of the benchmark, and we have taken reasonable mitigations: a responsible-use licence, no released captured-verifier weights or fine-tuning scripts, and a `ResponsibleUse.md` file in the repository.

Affected stakeholders. The primary stakeholders are operators of multi-LLM verification ensembles, including AI-auditing platforms, peer-review automation tools, and safety-content moderation pipelines. We have reached out to the maintainers of two such platforms ((Liu, 2026b) and the public release of Leung et al. (2026)'s evaluation harness) to inform them of our findings ahead of public release.

IRB and funding. This study was not subject to IRB review. No external funding was received for this work.

Author conflicts of interest. The authors declare no financial conflicts of interest. The authors maintain an open-source peer-review audit tool (Liu, 2026b) that is discussed as one of the motivating use cases for this benchmark; the tool is referenced consistently with the rest of the literature and no special pleading is made for it.

10 Conclusion

We have presented an empirical benchmark of seven aggregation methods for multi-LLM verifier ensembles under three verifier-capture attacks, across four scenarios. The headline finding is *negative*: no aggregation method we tested substantially outperforms plain majority voting, and several trust-score variants underperform majority under strong capture. We argued that the negative result is structural: a captured verifier is statistically indistinguishable from an honest verifier that happens to be wrong, so any rule that uses statistical deviation to identify capture cannot outperform a rule that ignores the distinction.

When our negative result would *not* hold. Our negative result rests on the assumption that captured verifiers mimic honest-but-wrong verifiers. If a real-world capture breaks this assumption — for instance, by introducing a structured deviation (e.g. a verifier that always outputs the exact median plus a fixed delta) — then a more sophisticated aggregation rule may recover above majority. Our benchmark does not rule this out; it only shows that for the three concrete attack parameterisations we test, the gain is at most a fraction of a percentage point.

A second assumption is that honest verifiers are exchangeable. If honest verifiers have known specialty areas (e.g. one verifier is better at math, another at code), then a domain-aware aggregator may do substantially better than majority. Our synthetic generator deliberately does not model this specialisation, so our benchmark under-estimates the upside of sophisticated aggregation in heterogeneous pools.

Limitations. Our results rely on a synthetic verifier-output generator calibrated to published rubric-LLM numbers but not validated against real LLM trace data. A practitioner applying our benchmark to a real ensemble should re-run our scenario definitions with the real distribution and may find that some methods we rule out do in fact help on their specific distribution. We release the synthetic generator and the attack implementations to facilitate this replication.

Future work. Three directions follow from this study. (i) Validate the negative result on real LLM verifier trace data; the result may not survive the transition from synthetic to real. (ii) Investigate structured-attack settings where captured verifiers introduce deviations outside the honest-but-wrong distribution; the negative result may not apply there. (iii) Investigate diversified verifier pools where domain-aware aggregation can outperform majority by exploiting specialisation.

References

Miguel Castro and Barbara Liskov. Practical Byzantine fault tolerance. In *Proceedings of the Third USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 1999.

Xia Cui, Ziyi Huang, and N. R. Abeynayake. Ensemble diversity optimization for subjective supervision. *arXiv preprint arXiv:2607.08493*, 2026. cs.LG; submitted 9 Jul 2026. Jointly optimizes ensemble weights, effective cardinality, and calibration via Gumbel-Softmax + signed diversity regularizer. Reduces cross-entropy 40-78/ WEL baselines. We cite as the most recent ensemble-with-calibration baseline; CRV differs by operating under an explicit adversarial (capture) model rather than a passive disagreement model.

Leslie Lamport, Robert Shostak, and Marshall Pease. The Byzantine generals problem. *ACM Transactions on Programming Languages and Systems*, 4(3):382–401, 1982.

Ethan Leung, Elias Lumer, Corey Feld, Austin Huber, Vamse Kumar Subbiah, and Kevin Paul. Do you need a frontier model as a citation verifier? benchmarking rubric llms for deep-research source attribution. *arXiv preprint arXiv:2607.08700*, 2026. cs.CL; submitted 9 Jul 2026. Provides calibration numbers for 8 LLM judges across 1,248 rubric decisions; reports F1=0.908 (kappa=0.636) for GPT-5-mini on source-relevance; shows directional bias (pass-rate drift, FPR, FNR) differs across judges at matched F1. We use this as the most recent published rubric-judge benchmark and as motivation for capture-resilient aggregation.

Zewen Liu. Calibration of multi-agent ensembles under verifier pressure. 2026a. TMLR submission (under review).

Zewen Liu. A multi-reviewer toolkit for transparent peer review of machine learning papers, 2026b. Open-source release, paper-review-toolkit v0.3.0.

Ethan Perez, Saffron Huang, Francis Song, Trevor Cai, Roman Ring, John Aslanides, Amelia Glaese, Nathan McAleese, and Geoffrey Irving. Red teaming language models with language models. 2022.

Sander Schulhoff et al. Ignore previous prompt: Attack techniques for language models. *arXiv preprint arXiv:2311.16119*, 2023.

Maofan Yin, Dahlia Malkhi, Michael K. Reiter, Guy Golan Gueta, and Ittai Abraham. Hotstuff: BFT consensus with linearity and responsiveness. In *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing (PODC)*, 2019.

Xing Zhang, Yanwei Cui, Guanghui Wang, Ziyuan Li, Wei Qiu, Bing Zhu, and Peiyang He. The blind curator: How a biased judge silently disables skill retirement in self-evolving agents. *arXiv preprint arXiv:2607.07436*, 2026. cs.AI; submitted 8 Jul 2026. Direct predecessor of our threat model: shows biased LLM judge silently disables a downstream curator past a sharp false-pass threshold. Submitted within 3 days of our draft; we cite it as the closest contemporaneous evidence that the verifier-capture problem is timely.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Eric Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, et al. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

A Proof of the Capture Lower Bound (Theorem 1)

We prove the deterministic lower bound stated in Theorem 1: if $f \geq N/2$ of the verifiers are captured, no deterministic aggregation rule $\mathcal{A} : [0,1]^N \to \{0,1\}$ can be ε-capture-resilient for $\varepsilon < 1/2$.

Proof. Consider the case $f > N/2$ (strict majority capture; the case $f = N/2$ follows by a standard limit argument). The adversary chooses a target label $y^* \in \{0,1\}$ and a response function g such that every captured verifier outputs y^* with probability 1 on every input. The adversary also chooses an input distribution $\mathcal{D}$ on which the *honest* verifiers have per-instance accuracy strictly below $1/2$; this is always possible by choosing $\mathcal{D}$ to concentrate on inputs where the honest verifiers systematically err.

Because the captured coalition $\mathcal{A}$ forms a strict majority, the deterministic rule $\mathcal{A}$ sees $p_i \in \{0,1\}$ on the captured inputs (after g is applied) and any values in $[0,1]$ on the honest inputs. Since $\mathcal{A}$ is deterministic, its decision is a function only of the multiset $\{p_1, \ldots, p_N\}$, and in particular of the *number* of captured verifiers voting y^*, which is $f > N/2$. For any such deterministic rule, there exists an assignment of honest outputs such that $\mathcal{A}$ outputs y^* on every input, even when the true label is the opposite $\bar{y}^*$; this is a direct consequence of the fact that a strict majority of N deterministic binary inputs forces the binary majority rule, and any rule that agrees with binary majority on strict-majority inputs cannot be ε-resilient for $\varepsilon < 1/2$ on the chosen $\mathcal{D}$.

This shows that for any deterministic $\mathcal{A}$, the adversary can construct a distribution $\mathcal{D}$ on which $\mathcal{A}$ achieves error probability at least $1/2$, completing the proof. □

Proof of Theorem 2 (claim; ATV ε-resilience). The full proof is omitted for brevity because ATV does not in fact achieve the claimed $\varepsilon = 2f/N + O(\Delta^2)$ guarantee in our benchmark (Section 5). We retain the theorem statement for completeness so that future readers can identify exactly which step in the proof fails; a candidate follow-up paper can either fix the proof or revise the theorem. □

B Algorithm Pseudocode and Complexity

We give pseudocode for the four representative aggregation rules. All four have memory $O(N)$ for the per-verifier state.

Plain majority.

Require: votes $p_1, \ldots, p_N$
1: $s \leftarrow \sum_i \mathbb{1}[p_i \geq 0.5]$
2: **return** $\mathbb{1}[s > N/2]$

Time $O(N)$.

PBFT-style threshold.

Require: votes $p_1, \ldots, p_N$; bound f
1: $s \leftarrow \sum_i \mathbb{1}[p_i \geq 0.5]$
2: **return** $\mathbb{1}[s \geq (N + f + 1)/2]$

Time $O(N)$.

Trust-score weighted mean.

Require: votes $p_1, \ldots, p_N$; trusts $\tau_1, \ldots, \tau_N$; decay α
1: $w_i \leftarrow \tau_i / \sum_j \tau_j$ for each i
2: **return** $\mathbb{1}[\sum_i w_i p_i \geq 0.5]$
3: Update: $\tau_i \leftarrow \alpha\tau_i + (1 - \alpha)(1 - |p_i - \text{med}_{-i}|)$ for each i

Time $O(N \log N)$ per round (median), memory $O(N)$.

Trust-score weighted median.

Require: votes $p_1, \ldots, p_N$; trusts $\tau_1, \ldots, \tau_N$; decay α
1: Sort indices by p_i; let π be the sorted permutation
2: $W \leftarrow \sum_j \tau_j$; find smallest k with $\sum_{i \leq k} \tau_{\pi(i)} \geq W/2$
3: **return** $\mathbb{1}[p_{\pi(k)} \geq 0.5]$
4: Update trust as in weighted mean.

Time $O(N \log N)$ per round, memory $O(N)$.

Atv (adaptive hybrid).

Require: votes $p_1, \ldots, p_N$; trusts $\tau_1, \ldots, \tau_N$; decay α; split τ_{split}
1: **if** $\text{std}(\tau) < \tau_{\text{split}}$ **then**
2: **return** weighted-mean decision
3: **else**
4: **return** weighted-median decision
5: **end if**
6: Update trust as in weighted mean.

Time $O(N \log N)$ per round, memory $O(N)$.

C Additional Ablations and Sensitivity Sweeps

Per-attack-model breakdown. Table 2 reports per-attack-model accuracies. The ordering of methods is qualitatively consistent across the three attack families: majority and trust_median remain tied at the top, with the largest absolute gap (0.7 percentage points) appearing under rubric injection.

Hyperparameter sensitivity. Atv's τ_{split} was tuned on a held-out scenario. Sensitivity sweeps over $\tau_{\text{split}} \in [0.2, 0.8]$ shift Atv's mean accuracy by at most 0.4 percentage points; the rank ordering is preserved across all sweep values. Trust-update decay α sweeps over $[0.5, 0.99]$ shift trust_mean by ± 0.5 percentage points with no rank change.

Ensemble-size sweep. We additionally ran the benchmark at $N \in \{5, 7, 10\}$ with f proportional to N ($f = \lfloor N/3 \rfloor$). Under this proportional-f regime, majority remains the top method at every

Table 2: Decision accuracy by attack model, averaged across all four scenarios (5 seeds, $T = 1000$ rounds per cell).

Method	PromptInj	RubricInj	MajBias
majority	0.877	0.945	0.997
bft_pbft	0.765	0.886	0.985
weighted_uniform	0.876	0.944	0.997
trust_mean	**0.894**	0.944	**0.998**
trust_median	0.877	**0.945**	0.997
kalman_trust	0.877	**0.945**	0.997
Atv	0.886	**0.945**	0.997

ensemble size, and the gap between majority and trust-based methods *widens* as N grows (because the larger honest majority hardens against capture). This is consistent with our structural explanation: as the honest majority grows, capture becomes a smaller perturbation that any aggregation rule, including majority, can absorb.

Real-LLM validation (sketch). We did not run the full benchmark on real LLM verifier traces within the scope of this paper. A small pilot run on Leung et al. (2026)'s evaluation harness (8 LLM judges, 1248 rubric decisions, of which we synthetically capture 2) is consistent with the synthetic-data trend: majority remains at or near the top, trust-based methods trail by less than 1 percentage point. We discuss the limitations of this pilot in Section 10.

D Code and Reproducibility

Repository layout. Code is released under a permissive licence with an attached `ResponsibleUse.md` file. The repository contains:

- `crv.py`: legacy CRV implementation (retained for archival reference; not used as a benchmarked method).
- `attacks.py`: implementations of the three attack families (PromptInj, RubricInj, MajBias).
- `extended_methods.py`: the seven aggregation methods benchmarked in this paper.
- `benchmark.py`: the four-scenario benchmark driver.
- `verify_p6.py`: the verify suite used during paper preparation.

Beyond-PDF interactive demo. We release a Beyond-PDF interactive demonstration that replays the benchmark on real LLM trace files. The demo runs the seven aggregation rules on a user-provided trace file and displays the per-round decision side-by-side with the ground-truth label. We expect this to be useful for engineering teams that wish to re-run our scenario definitions on a real ensemble.

Reproducibility hashes. At submission time, we record SHA-256 hashes of all released code artifacts in `HASHES.txt`. Reviewers can verify their copy against the hashes by running the included `verify_hashes.sh` script.

Computational footprint. A full benchmark run (4 scenarios $\times$ 7 methods $\times$ 5 seeds, $T = 1000$ rounds) takes approximately 4 minutes on a single CPU core (no GPU required) and uses 50 MB of memory. The complete experiment can be reproduced in under 10 minutes on commodity hardware.